

Hierarchical Memory for Long-Horizon LLMs **Core Idea**

Move old context from expensive full KV state into cheaper compressed, symbolic, summary, or graph memory, then reconstruct or retrieve details only when needed. 1. Long-context Transformer Memory **Transformer-XL**

Reuses hidden states from previous segments using recurrence. Early version of “keep past neural state around.”

Compressive Transformer

Keeps recent memory at high fidelity and compresses older memory. Very close to a raw KV → compressed memory hierarchy.

Memorizing Transformers

Stores past key-value-like representations in external kNN memory. Closest to retrieving neural state later.

Takeaway: These works prove hierarchical neural memory can work, but do not fully solve agentic million-turn memory management. 2. Retrieval-Augmented LLMs **REALM**

Trains a language model with a latent document retriever.

RETRO

Retrieves chunks from a massive database and cross-attends to them.

Classic RAG

Retrieves relevant text chunks into the prompt.

Takeaway: RAG retrieves text, but a structured hierarchy can preserve relationships and reasoning paths better than flat retrieval. 3. Agent Memory Systems **MemGPT**

Treats LLM context like virtual memory, moving information between memory tiers.

Letta

Production evolution of MemGPT with core memory, archival memory, recursive summaries, and recall search.

Generative Agents

Stores observations, synthesizes reflections, and retrieves memories for planning.

MEMORYLLM

Adds a latent memory pool that can self-update over time.

Takeaway: Agent-memory systems already use hierarchy, but mostly over text memories, not full KV states. 4. Graph / Knowledge-Tree Memory **GraphRAG**

Builds knowledge graphs, summarizes communities, and retrieves through graph structure.

Graph-based Agent Memory

Stores entities, relationships, events, and summaries.

Knowledge-Augmented Generation

Uses structured knowledge graphs for retrieval and reasoning.

Takeaway: GraphRAG is one of the closest public directions to concept-tree memory, but it does not preserve or reconstruct KV. 5. KV Cache Compression and Eviction **ChunkKV**

Compresses KV by preserving semantically informative chunks.

ShotKV

Handles prefill and decoding differently while preserving semantic coherence.

RocketKV

Two-stage KV compression with coarse eviction and sparse attention.

HCAAttention

Combines quantization, value offload, and dynamic KV eviction.

Takeaway: KV compression is advanced, but most methods decide importance from attention or statistics rather than semantic hierarchy. The Gap Your Idea Targets Current systems typically do one of the following: Raw KV -> Compression

RAG -> Text Retrieval

Agent Memory -> Conversation Summaries

GraphRAG -> Concept Graphs

Your idea combines all of them: Raw KV

↓

Compressed KV

↓

Semantic Chunk Summary

↓

Concept Node

↓

Knowledge Tree / Graph

↓

Selective Reconstruction Best Research Framing **Hierarchical Semantic KV Memory for**

Long-Horizon Agentic LLM Inference Central Question: Can an LLM demote old context from full KV into compressed semantic memory, preserve pointers to raw text/KV, and later reconstruct only the pieces needed for reasoning? Why This Matters for Hardware Traditional memory cost grows approximately with all prior tokens times KV size. Your direction aims to make cost scale with active reasoning state plus compressed semantic memory. The long-term architecture resembles: L1/L2 Cache -> HBM KV -> CPU KV -> SSD KV -> Summaries -> Graph Memory The most promising opportunity is not merely compressing KV further, but building hardware and runtime support that move information between neural KV, compressed KV, text memory, and concept-graph memory.